

محاضرة حاسوبية
هندسة ذكاء ثالثة

Registers

- A register is a group of flip-flops (typically D-type) that share a common clock and are capable of storing binary data. Each flip-flop stores one bit, so an n-bit register has n flip-flops.
- A register is a temporary storage area built into a CPU. Some registers are used internally and cannot be accessed outside the processor, while others are user-accessible. Most modern CPU architectures include both types of registers.

1. Internal registers includes:

- instruction register (IR)
- Memory buffer Register (MBR)
- Memory data register (MDR)
- Memory address register (MAR).
- ~~Instruction register fetches instructions from the~~
- Program counter (PC) and holds each instruction as it is executed by the Processor.
- Memory registers are used to pass data from memory to the processor.

عدد
البرنامج

The storage time of internal registers is extremely temporary, as they often hold data for less than a millisecond.

2. User-accessible registers:

They are larger than internal registers and typically hold data for a longer time. For example, a data register may store individual values referenced being by a currently running program. An address register contains memory addresses, which reference different blocks of memory within the system RAM. Many CPUs now have general purpose registers (GPRs), which may contain both data and memory addresses.

Registers vary in both number and size, depending on the CPU architecture.

Some processors have 8 registers while others have 16, 32, or more. For many years, registers were 32-bit, but now many are 64-bit in size. A 64-bit register is necessary for a 64-bit processor, since it enables the CPU to access 64-bit memory addresses. A 64-bit register can also store 64-bit instructions, which cannot be loaded into a 32-bit register. Therefore, most programs written for 32-bit processors can run on 64-bit computers, while 64-bit programs are not backwards compatible with 32-bit machines.

Registers types:

1- General purpose registers: 64 R15

Registers are the most important components of CPU. Each register performs a specific function. A brief description of most important CPU's registers and their functions are given below:

1. Memory Address Register (MAR):

This register holds the address of memory where CPU wants to read or write data. When CPU wants to

store some data in the memory or reads the data from the memory, it places the address of the required memory location in the MAR.

2. Memory Buffer Register (MBR):

This register holds the contents of data or instruction read from, or written in memory. The contents of instruction placed in this register are transferred to the Instruction Register, while the contents of data are

المجمع
transferred to the accumulator or I/O register. In other words you can say that this register is used to store

data/instruction coming from the memory or going to the memory.

3. I/O Address Register (I/O AR):

I/O Address register is used to specify the address of a particular I/O device.

4. I/O Buffer Register (I/O BR):

I/O Buffer Register is used for exchanging data between the I/O module and the processor.

5. Program Counter (PC)

Program Counter register is also known as Instruction Pointer Register. This register is used to store the address of the next instruction to be fetched for execution. When the instruction is fetched, the value of IP is

تمت

زيادته

incremented. Thus this register always points or holds the address of next instruction to be fetched.

6. Instruction Register (IR):

Once an instruction is fetched from main memory, it is stored in the Instruction Register. The control unit takes instruction from this register, decodes and executes it by sending signals to the appropriate component of computer to carry out the task.

7. Accumulator Register (AX):

المخزن

The accumulator register is located inside the ALU, It is used during arithmetic & logical operations of ALU. The control unit stores data values fetched from main memory in the accumulator for arithmetic or logical operation. This register holds the initial data to be operated upon, the intermediate results, and the final result of operation. The final result is transferred to main memory through MBR.

8. Stack Control Register:

A stack represents a set of memory blocks; the data is stored in and retrieved from these blocks in an order, i.e. First In and Last Out (FILO). The Stack Control Register is used to manage the stacks in memory. The size of this register is 2 or 4 bytes.

9. Flag Register (FR):

حدوث
يشير

The Flag register is used to indicate occurrence of a certain condition during an operation of the CPU. It is a special purpose register with size one byte or two bytes. Each bit of the flag register constitutes a flag (or alarm), such that the bit value indicates if a specified condition was encountered while executing an instruction. For example, if zero value is put into an arithmetic register (accumulator) as a result of an arithmetic operation or a comparison, then the zero flag will be raised by the CPU. Thus, the subsequent instruction can check this flag and when a zero flag is "ON" it can take, an appropriate route in the algorithm.

2-Segment registers:

Segmentation is the process in which the main memory of the computer is logically divided into different segments and each segment has its own base address. It is basically used to enhance the speed of execution of the computer system, so that the processor is able to fetch and execute the data from the memory easily and fast. We create segments in memory (segmentation), To get the starting location of this segments we store the address of this segments in segment register. In short segment registers store the starting location of segments present in memory i.e segment address.

Need for Segmentation

The Bus Interface Unit (BIU) contains four 16 bit special purpose registers (mentioned below) called as Segment Registers.

- **Code segment register (CS):** is used for addressing memory location in the code segment of the memory, where the executable program is stored.
- **Data segment register (DS):** points to the data segment of the memory where the data is stored.
- **Extra Segment Register (ES):** also refers to a segment in the memory which is another data segment in the memory.
- **Stack Segment Register (SS):** is used for addressing stack segment of the memory. The stack segment is that segment of memory which is used to store stack data.

3-Index Register

A register that can be specified by instructions that use indexed addressing. An index register is usually controlled by one or more instructions with the ability to increment or decrement the register by a fixed amount, to test the register for equality with a specified value (often zero), and to jump to a specified location when equality is achieved. It may be part of the processor status word.

4- Status and Control register

Control and Status Register - (CSR) A register in most CPUs which stores additional information about the results of machine instructions, e.g. comparisons. It usually

consists of several independent flags such as carry, overflow and zero. The CSR is chiefly used to determine the outcome of conditional branch instructions or other forms of conditional execution. (logical) The interface of an I/O device includes a control register that contains the information that governs the mode of operation of the device.

One bit in this register may be dedicated to interrupt control. The I/O device is allowed to raise interrupt requests only when this bit is set to 1.

For review

What is relation between Status register and Control register?

The information needed to determine whether a device is requesting an interrupt is available in its status register. When the device raises an interrupt request, it sets to 1 a bit in its status register, which we will call the IRQ bit. The simplest way to identify the interrupting device is to have the interrupt-service routine poll all I/O devices in the system. The first device encountered with its IRQ bit set to 1 is the device that should be serviced. An appropriate subroutine is then called to provide the requested service.

For Carry

	BC_{in}	00	01	11	10
A	0	0	0	1	0
	1	0	1	1	1

$$\text{Carry, } C_{out} = AB + AC_{in} + BC_{in}$$

For Sum

	BC_{in}	00	01	11	10
A	0	0	1	0	1
	1	1	0	1	0

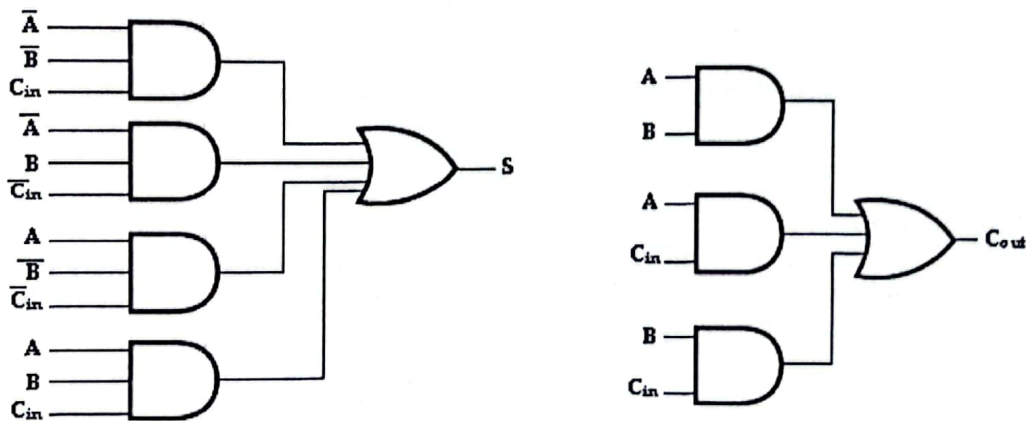
$$\text{Sum, } S = A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in}$$

The Boolean expressions for the SUM and CARRY outputs are given by the equations,

$$\text{Sum, } S = A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in}$$

$$\text{Carry, } C_{out} = AB + AC_{in} + BC_{in}$$

The logic diagram for the above functions is shown as,



Implementation of full-adder in Sum of Products

The logic diagram of the full adder can also be implemented with two half-adders and one OR gate. The S output from the second half adder is the exclusive-OR of C_{in} and the output of the first half-adder, giving

$$\begin{aligned} \text{Sum} &= C_{in} \oplus (A \oplus B) \\ &= C_{in} \oplus (A'B + AB') \\ &= C'_{in} (A'B + AB') + C_{in} (A'B + AB')' \\ &= C'_{in} (A'B + AB') + C_{in} (AB + A'B') \\ &= A'BC'_{in} + AB'C'_{in} + ABC_{in} + A'B'C_{in} \end{aligned}$$

$$[x \oplus y = x'y + xy']$$

$$[(x'y + xy')' = (xy + x'y')]$$

and the carry output is,

$$\text{Carry, } C_{out} = AB + C_{in} (A'B + AB')$$

$$= AB + A'BC_{in} + AB'C_{in}$$

$$= AB(C_{in}+1) + A'BC_{in} + AB'C_{in}$$

$$[C_{in}+1=1]$$

$$= ABC_{in} + AB + A'BC_{in} + AB'C_{in}$$

$$= AB + AC_{in}(B+B') + A'BC_{in}$$

$$= AB + AC_{in} + A'BC_{in}$$

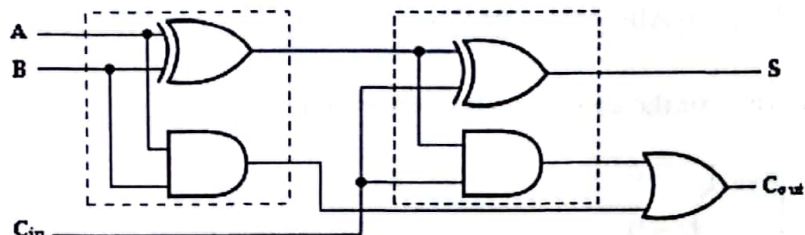
$$= AB(C_{in}+1) + AC_{in} + A'BC_{in}$$

$$[C_{in}+1=1]$$

$$= ABC_{in} + AB + AC_{in} + A'BC_{in}$$

$$= AB + AC_{in} + BC_{in}(A+A')$$

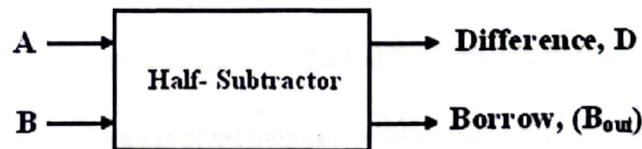
$$= AB + AC_{in} + BC_{in}$$



Implementation of full adder with two half-adders and an OR gate

Half -Subtractor:

A *half-subtractor* is a combinational circuit that can be used to subtract one binary digit from another to produce a DIFFERENCE output and a BORROW output. The BORROW output here specifies whether a '1' has been borrowed to perform the subtraction.



Block schematic of half-subtractor

The truth table of half-subtractor, showing all possible input combinations and the corresponding outputs are shown below.

Input		Output	
A	B	Difference (D)	Borrow (B _{out})
0	0	0	0
0	1	1	1
1	0	1	0